

Object Detection Extension

<https://github.com/aiyshwary/Object-Detection-Extension-in-Rapid-Miner>

[Python](#)

[PyTorch](#)

[ONNX Runtime](#)

[RapidMiner](#)

[Object Detection](#)

OVERVIEW

A RapidMiner Studio extension that brings transfer learning-based object detection into a no-code environment, supporting fine-tuning and inference with Faster R-CNN, FCOS, SSD, SSDLite, and RetinaNet models.

IMPACT

Developed two RapidMiner operators for object detection — `FineTuneObjectDetectionModel` and `ObjectDetectionModelInference` — enabling users to train and deploy detection models within a visual workflow without writing code.

TECHNICAL WALKTHROUGH

Problem Statement

Different object detection use cases require different trade-offs between accuracy, speed, and computational cost. A single model cannot satisfy all scenarios.

Approach

Developed a unified object detection extension using PyTorch, supporting multiple architectures:

i) Faster R-CNN

ii) FCOS

iii) SSD

iv) SSD Lite

v) RetinaNet

vi) YOLO

Training

i) Dataset annotated using XML bounding boxes

ii) Transfer learning using COCO pre-trained weights

Fully configurable training pipeline

i) Model selection

ii) Batch size

iii) Epochs

iv) Learning rate

v) Momentum

vi) Weight decay

Evaluation metrics

i) IoU

ii) Precision

iii) Recall

Inference

Users can choose any trained or pre-trained model

Output includes

i) Image with bounding boxes, labels, and confidence scores

ii) Tabular output with detected objects

iii) Results also generated in base64 format for easy deployment and integration

Key Observations

i) Faster R-CNN performed best for accuracy

ii) SSD and SSD Lite were faster but had lower recall

iii) RetinaNet handled class imbalance well

iv) Model choice depends on application requirements

KEY DETAILS

1. FineTuneObjectDetectionModel accepts image and annotation directories (YOLO/Pascal VOC) and outputs model weights and class list CSV.

2. ObjectDetectionModelInference runs predictions on new images and returns bounding boxes and class labels as a DataFrame.

3. Supports five architectures: Faster R-CNN, FCOS, SSD, SSDLite, and RetinaNet for varying speed-accuracy trade-offs.

4. Configurable for CPU-only and GPU (CUDA cu118) environments; installed as a JAR extension in RapidMiner Studio.

5. Enables domain-specific object detection with minimal ML expertise using a drag-and-drop process designer.

6. Inference operator can optionally save annotated images with overlaid prediction bounding boxes via Matplotlib, in addition to the DataFrame output.

7. Uses chardet for automatic annotation-file encoding detection, ensuring robust parsing of YOLO/Pascal VOC label files across different systems and locales.